

Résumé Complet Redis

CRUD, Structures de Données, Commandes

AmineGR03

1^{er} février 2026

Table des matières

Opérations CRUD de Base

String (Chaîne)

Create (SET)

```
1 SET cle "valeur"
2 SET nom "Dupont"
3 SET age 30
```

Read (GET)

```
1 GET cle
2 GET nom
3 GET age
```

Update (SET)

```
1 SET nom "Martin" -- Remplace la valeur
```

Delete (DEL)

```
1 DEL cle
2 DEL nom age -- Supprime plusieurs clés
```

Opérations Avancées sur String

```
1 -- SET avec expiration (secondes)
2 SET cle "valeur" EX 3600
3
4 -- SET avec expiration (millisecondes)
5 SET cle "valeur" PX 3600000
6
7 -- SET si n'existe pas (NX)
8 SET cle "valeur" NX
9
10 -- SET si existe (XX)
11 SET cle "valeur" XX
12
13 -- GETSET (récupère et remplace)
14 GETSET cle "nouvelle_valeur"
15
16 -- APPEND
17 APPEND cle "_suffixe"
18
19 -- INCR (incrémenter)
20 INCR compteur
21
22 -- INCRBY (incrémenter de n)
23 INCRBY compteur 5
24
25 -- DECR (décrémenter)
26 DECR compteur
```

```
27
28 -- DECRBY (d cr menter de n)
29 DECRBY compteur 3
30
31 -- MGET (multiple GET)
32 MGET cle1 cle2 cle3
33
34 -- MSET (multiple SET)
35 MSET cle1 "valeur1" cle2 "valeur2" cle3 "valeur3"
```

Hash (Tableau Associatif)

Create (HSET)

```
1 HSET utilisateur:1 nom "Dupont"
2 HSET utilisateur:1 age 30
3 HSET utilisateur:1 ville "Paris"
```

Read (HGET, HGETALL)

```
1 HGET utilisateur:1 nom
2 HGETALL utilisateur:1
```

Update (HSET)

```
1 HSET utilisateur:1 age 31
```

Delete (HDEL)

```
1 HDEL utilisateur:1 ville
2 HDEL utilisateur:1 nom age -- Supprime plusieurs champs
```

Opérations Avancées sur Hash

```
1 -- HMSET (multiple SET)
2 HMSET utilisateur:1 nom "Dupont" age 30 ville "Paris"
3
4 -- HMGET (multiple GET)
5 HMGET utilisateur:1 nom age ville
6
7 -- HKEYS (toutes les cl s)
8 HKEYS utilisateur:1
9
10 -- HVALS (toutes les valeurs)
11 HVALS utilisateur:1
12
13 -- HEXISTS (v rifier existence)
14 HEXISTS utilisateur:1 nom
15
16 -- HLEN (nombre de champs)
17 HLEN utilisateur:1
18
```

```

19 -- HINCRBY (incr menter champ num rique)
20 HINCRBY utilisateur:1 age 1
21
22 -- HINCRBYFLOAT
23 HINCRBYFLOAT utilisateur:1 score 0.5

```

List (Liste)

Create (LPUSH, RPUSH)

```

1 LPUSH liste "element1"
2 LPUSH liste "element2" "element3"
3 RPUSH liste "element4"

```

Read (LRANGE, LINDEX)

```

1 LRANGE liste 0 -1 -- Tous les lments
2 LRANGE liste 0 2 -- 3 premiers lments
3 LINDEX liste 0 -- Premier lment
4 LINDEX liste -1 -- Dernier lment

```

Update (LSET)

```

1 LSET liste 0 "nouvelle_valeur"

```

Delete (LPOP, RPOP, LREM)

```

1 LPOP liste -- Retire le premier
2 RPOP liste -- Retire le dernier
3 LREM liste 2 "val" -- Retire 2 occurrences de "val"

```

Opérations Avancées sur List

```

1 -- LLEN (longueur)
2 LLEN liste
3
4 -- LTRIM (garder seulement un range)
5 LTRIM liste 0 2
6
7 -- RPOPLPUSH (d placer de fin d but d'une autre liste)
8 RPOPLPUSH liste1 liste2
9
10 -- BLPOP (blocking left pop)
11 BLPOP liste 10 -- Attend jusqu' 10 secondes
12
13 -- BRPOP (blocking right pop)
14 BRPOP liste 10

```

Set (Ensemble)

Create (SADD)

```
1 SADD tags "important"
2 SADD tags "urgent" "prioritaire"
```

Read (SMEMBERS, SISMEMBER)

```
1 SMEMBERS tags
2 SISMEMBER tags "important" -- Retourne 1 si existe, 0 sinon
```

Update (SADD pour ajouter)

```
1 SADD tags "nouveau_tag"
```

Delete (SREM)

```
1 SREM tags "important"
2 SREM tags "urgent" "prioritaire"
```

Opérations Avancées sur Set

```
1 -- SCARD (cardinalit )
2 SCARD tags
3
4 -- SRANDMEMBER ( lment al atoire)
5 SRANDMEMBER tags
6 SRANDMEMBER tags 3 -- 3 lments al atoirs
7
8 -- SPOP (retirer un lment al atoire)
9 SPOP tags
10
11 -- SDIFF (diff rence)
12 SDIFF set1 set2
13
14 -- SINTER (intersection)
15 SINTER set1 set2
16
17 -- SUNION (union)
18 SUNION set1 set2
19
20 -- SDIFFSTORE (diff rence et stocker)
21 SDIFFSTORE resultat set1 set2
22
23 -- SINTERSTORE (intersection et stocker)
24 SINTERSTORE resultat set1 set2
25
26 -- SUNIONSTORE (union et stocker)
27 SUNIONSTORE resultat set1 set2
28
29 -- SMOVE (d placer lment )
30 SMOVE set1 set2 "element"
```

Sorted Set (Ensemble Ordonné)

Create (ZADD)

```
1 ZADD scores 100 "joueur1"
2 ZADD scores 200 "joueur2" 150 "joueur3"
```

Read (ZRANGE, ZREVRANGE)

```
1 ZRANGE scores 0 -1          -- Tous, par score croissant
2 ZRANGE scores 0 -1 WITHSCORES -- Avec scores
3 ZREVRANGE scores 0 -1       -- Par score d croissant
4 ZREVRANGE scores 0 2        -- Top 3
5 ZSCORE scores "joueur1"     -- Score d'un membre
```

Update (ZADD pour modifier score)

```
1 ZADD scores 250 "joueur1" -- Met jour le score
2 ZINCRBY scores 10 "joueur1" -- Incr mente le score
```

Delete (ZREM)

```
1 ZREM scores "joueur1"
2 ZREM scores "joueur2" "joueur3"
```

Opérations Avancées sur Sorted Set

```
1 -- ZCARD (nombre d' lments )
2 ZCARD scores
3
4 -- ZCOUNT (compter dans un range de scores)
5 ZCOUNT scores 100 200
6
7 -- ZRANK (rang d'un membre, croissant)
8 ZRANK scores "joueur1"
9
10 -- ZREVRANK (rang d'un membre, d croissant)
11 ZREVRANK scores "joueur1"
12
13 -- ZRANGEBYSCORE (par range de scores)
14 ZRANGEBYSCORE scores 100 200
15 ZRANGEBYSCORE scores 100 200 WITHSCORES LIMIT 0 10
16
17 -- ZREVRANGEBYSCORE
18 ZREVRANGEBYSCORE scores 200 100
19
20 -- ZREMRANGEBYRANK (supprimer par rang)
21 ZREMRANGEBYRANK scores 0 2 -- Supprime les 3 premiers
22
23 -- ZREMRANGEBYSCORE (supprimer par score)
24 ZREMRANGEBYSCORE scores 0 100
25
26 -- ZUNIONSTORE (union de sorted sets)
```

```
27 ZUNIONSTORE resultat 2 set1 set2 WEIGHTS 1 2
28
29 -- ZINTERSTORE (intersection de sorted sets)
30 ZINTERSTORE resultat 2 set1 set2
```

Gestion des Clés

Opérations sur Clés

```

1 -- EXISTS (v rifier existence)
2 EXISTS cle
3
4 -- TYPE (type de la cl )
5 TYPE cle
6
7 -- TTL (time to live en secondes)
8 TTL cle
9
10 -- PTTL (time to live en millisecondes)
11 PTTL cle
12
13 -- EXPIRE (d finir expiration en secondes)
14 EXPIRE cle 3600
15
16 -- PEXPIRE (d finir expiration en millisecondes)
17 PEXPIRE cle 3600000
18
19 -- EXPIREAT (expiration un timestamp)
20 EXPIREAT cle 1234567890
21
22 -- PERSIST (supprimer expiration)
23 PERSIST cle
24
25 -- RENAME (renommer)
26 RENAME ancienne nouvelle
27
28 -- RENAMENX (renommer si nouvelle n'existe pas)
29 RENAMENX ancienne nouvelle
30
31 -- KEYS (trouver toutes les cl s correspondant au pattern)
32 KEYS utilisateur:*
33
34 -- SCAN (scanner les cl s , recommand )
35 SCAN 0 MATCH utilisateur:* COUNT 100
36
37 -- DEL (supprimer)
38 DEL cle1 cle2 cle3
39
40 -- UNLINK (supprimer de mani re asynchrone)
41 UNLINK cle1 cle2

```

Patterns de Clés

```

1 -- * : correspond n'importe quel caract re
2 KEYS utilisateur:*
3
4 -- ? : correspond un seul caract re
5 KEYS utilisateur:?
6
7 -- [abc] : correspond a, b ou c
8 KEYS utilisateur:[123]
9

```



```
10 -- [a-z] : correspond un caract re dans la plage  
11 KEYS utilisateur:[1-9]
```

Transactions

MULTI/EXEC

```
1 MULTI
2 SET cle1 "valeur1"
3 SET cle2 "valeur2"
4 INCR compteur
5 EXEC
```

DISCARD

```
1 MULTI
2 SET cle1 "valeur1"
3 DISCARD -- Annule la transaction
```

WATCH

```
1 WATCH cle
2 MULTI
3 SET cle "valeur"
4 EXEC -- choue si cle a t modifi e entre WATCH et EXEC
```

Pub/Sub (Publication/Souscription)

PUBLISH

```
1 PUBLISH canal "message"
```

SUBSCRIBE

```
1 SUBSCRIBE canal1 canal2
```

PSUBSCRIBE (Pattern Subscribe)

```
1 PSUBSCRIBE news.*
```

UNSUBSCRIBE

```
1 UNSUBSCRIBE canal1
```

PUNSUBSCRIBE

```
1 PUNSUBSCRIBE news.*
```

Exemples Pratiques

Exemple 1 : Système de Votes

```
1 -- Voter pour un article
2 ZINCRBY article:votes 1 "article:123"
3
4 -- Obtenir les top articles
5 ZREVRANGE article:votes 0 9 WITHSCORES
6
7 -- Vérifier le score d'un article
8 ZSCORE article:votes "article:123"
```

Exemple 2 : Cache Utilisateur

```
1 -- Stocker un utilisateur
2 HMSET utilisateur:123 nom "Dupont" age 30 ville "Paris"
3 EXPIRE utilisateur:123 3600
4
5 -- Récupérer un utilisateur
6 HGETALL utilisateur:123
7
8 -- Mettre à jour
9 HSET utilisateur:123 age 31
```

Exemple 3 : File d'Attente

```
1 -- Ajouter à la file
2 LPUSH file:travaux "travail1"
3 LPUSH file:travaux "travail2"
4
5 -- Traiter un travail
6 RPOP file:travaux
7
8 -- Voir la file
9 LRange file:travaux 0 -1
```

Exemple 4 : Tags d'Articles

```
1 -- Ajouter des tags à un article
2 SADD article:123:tags "tech" "programming" "redis"
3
4 -- Trouver tous les articles avec un tag
5 SMEMBERS article:123:tags
6
7 -- Trouver les articles communs (intersection)
8 SINTER article:123:tags article:456:tags
```

Exemple 5 : Leaderboard

```
1 -- Ajouter un score
2 ZADD leaderboard 1000 "joueur1"
3 ZADD leaderboard 2000 "joueur2"
4 ZADD leaderboard 1500 "joueur3"
5
6 -- Top 10
7 ZREVRANGE leaderboard 0 9 WITHSCORES
8
9 -- Rang d'un joueur
10 ZREVRANK leaderboard "joueur1"
11
12 -- Incr menter le score
13 ZINCRBY leaderboard 100 "joueur1"
```

Résumé des Opérations Essentielles

Structures de Données

Type	Create	Read
String	SET	GET
Hash	HSET	HGET, HGETALL
List	LPUSH, RPUSH	LRANGE, LINDEX
Set	SADD	SMEMBERS
Sorted Set	ZADD	ZRANGE, ZREVRANGE

Opérations CRUD par Type

Type	Create	Read	Update	Delete
String	SET	GET	SET	DEL
Hash	HSET	HGET	HSET	HDEL
List	LPUSH/RPUSH	LRANGE	LSET	LPOP/RPOP
Set	SADD	SMEMBERS	SADD	SREM
Sorted Set	ZADD	ZRANGE	ZADD	ZREM

Commandes Utiles

Commande	Description
EXISTS	Vérifier existence d'une clé
TYPE	Type d'une clé
TTL	Time to live
EXPIRE	Définir expiration
KEYS	Trouver des clés (pattern)
SCAN	Scanner les clés (recommandé)
DEL	Supprimer une clé
MULTI/EXEC	Transaction
WATCH	Surveiller une clé

Points Importants

- **String** : Valeur simple, peut être n'importe quel type
- **Hash** : Structure clé-valeur imbriquée, idéal pour objets
- **List** : Liste ordonnée, peut être utilisée comme file
- **Set** : Ensemble non ordonné, opérations d'ensemble
- **Sorted Set** : Ensemble ordonné par score, idéal pour leaderboards
- **TTL** : Toutes les structures peuvent avoir une expiration
- **Transactions** : MULTI/EXEC pour exécuter plusieurs commandes atomiquement
- **Pub/Sub** : Système de messagerie asynchrone
- **KEYS vs SCAN** : SCAN est recommandé pour la production (non-bloquant)